國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab07

姓名：林彥廷

學號：112370219

Lab01-Q1.
Convert Infix Expression to Postfix Using a Stack Implemented with Linked List
Code

```cpp
#include <iostream>
#include <cctype>
#include <cstring>
using namespace std;

// 定義 Stack 的節點結構
struct Node {
    char data;  // 存儲字符 (運算子或括號)
    Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
    Node* top; // 指向堆疊頂端
public:
    Stack() { top = nullptr; } // 初始化堆疊

    // Push 操作：將元素放入堆疊
    void push(char ch) {
        Node* newNode = new Node; // 創建新節點
        newNode->data = ch;       // 存入數據
        newNode->next = top;      // 讓新節點指向目前的頂端
        top = newNode;            // 更新頂端指標
    }

    // Pop 操作：移除並回傳頂端元素
    char pop() {
        if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
        char ch = top->data;         // 取得頂端元素
        Node* temp = top;            // 暫存頂端節點
        top = top->next;             // 移動頂端指標
        delete temp;                 // 釋放記憶體
        return ch;                   // 回傳彈出的元素
```

```
    }

    // Peek 操作：取得頂端元素但不移除
    char peek() {
        return (top ? top->data : '\0');
    }

    // 判斷堆疊是否為空
    bool isEmpty() {
        return top == nullptr;
    }
};

// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
    // 設定乘除的優先值為 2，加減的優先值為 1
    if (op == '*' or op == '/') {
        return 2;
    }
    else if (op == '+' or op == '-') {
        return 1;
    }
    else {
        return 0;
    }
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
    Stack s;
    int k = 0;

    for (int i = 0; i < strlen(infix); i ++) { // 遍歷每個 infix 元素

        if (isalnum(infix[i])) {
            postfix[k++] = infix[i]; //如果是字母或數字就放到 postfix 裡，然後 K 往
後一格
        }
```

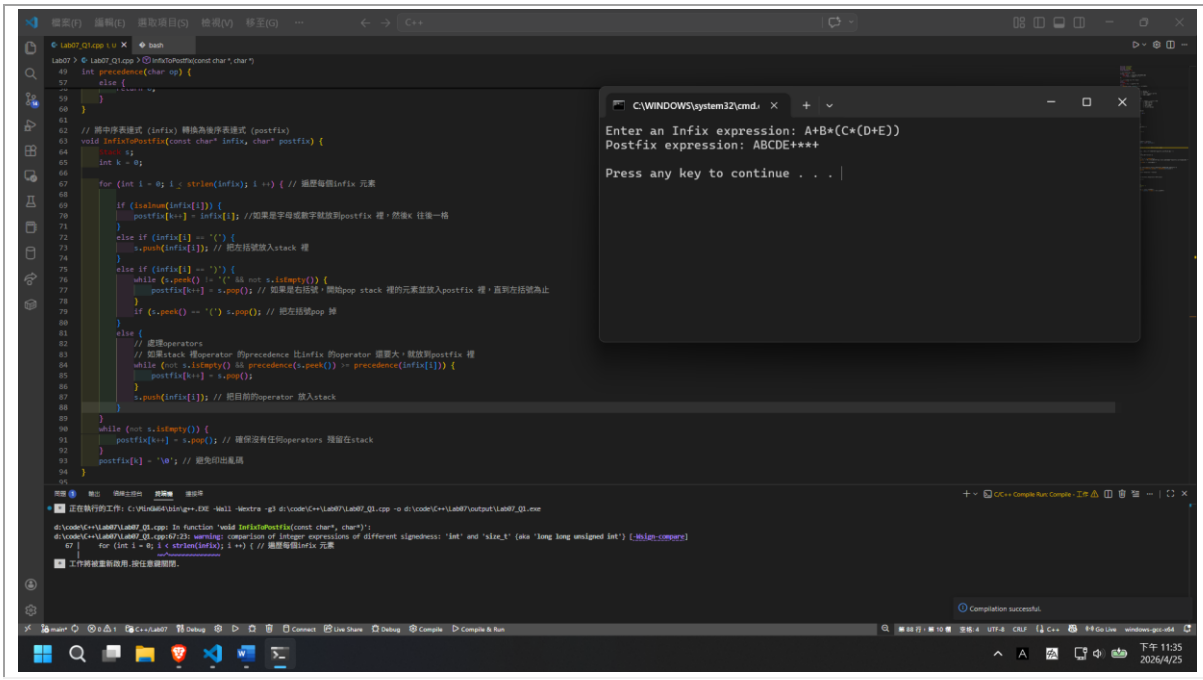
```
        else if (infix[i] == '(') {
            s.push(infix[i]); // 把左括號放入 stack 裡
        }
        else if (infix[i] == ')') {
            while (s.peek() != '(' && not s.isEmpty()) {
                postfix[k++] = s.pop(); // 如果是右括號，開始 pop stack 裡的元素並
放入 postfix 裡，直到左括號為止
            }
            if (s.peek() == '(') s.pop(); // 把左括號 pop 掉
        }
        else {
            // 處理 operators
            // 如果 stack 裡 operator 的 precedence 比 infix 的 operator 還要大，就放
到 postfix 裡
            while (not s.isEmpty() && precedence(s.peek()) >=
precedence(infix[i])) {
                postfix[k++] = s.pop();
            }
            s.push(infix[i]); // 把目前的 operator 放入 stack
        }
    }
    while (not s.isEmpty()) {
        postfix[k++] = s.pop(); // 確保沒有任何 operators 殘留在 stack
    }
    postfix[k] = '\0'; // 避免印出亂碼
}

int main() {
    char infix[100], postfix[100];
    cout << "Enter an Infix expression: ";
    cin >> infix; // 輸入中序表達式

    InfixToPostfix(infix, postfix); // 轉換為後序表達式
    cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式

    return 0;
}
```

Result



Discussion

TOKEN	STACK [0]	[1]	[2]	[3]	[4]	TOP	OUTPUT
(	(					0	
A	(					0	A
+	(	+				1	A
B	(	+				1	AB
)						-1	AB+
*	*					0	AB+
D	*					0	AB+D
+	+					0	AB+D*
E	+					0	AB+D*E
/	+	/				1	AB+D*E
(	+	/	(			2	AB+D*E
F	+	/	(			2	AB+D*EF
+	+	/	(	+		3	AB+D*EF
A	+	/	(	+		3	AB+D*EFA
*	+	/	(	+	*	4	AB+D*EFA
D	+	/	(	+	*	4	AB+D*EFAD
)	+	/				1	AB+D*EFAD*+
+	+					0	AB+D*EFAD*+/+
C	+					0	AB+D*EFAD*+/+C
<eos>						-1	AB+D*EFAD*+/+C+

TOKEN	STACK [0]	[1]	[2]	[3]	[4]	[5]	TOP	OUTPUT
A							-1	A
+	+						0	A
B	+						0	AB
*	+	*					1	AB
(	+	*	(				2	AB
C	+	*	(				2	ABC
*	+	*	(	*			3	ABC
(	+	*	(	*	(		4	ABC
D	+	*	(	*	(		4	ABCD
+	+	*	(	*	(	+	5	ABCD
E	+	*	(	*	(	+	5	ABCDE
)	+	*	(	*			3	ABCDE+
)	+	*					1	ABCDE+*
<eos>							-1	ABCDE+**+

TOKEN	STACK [0]	[1]	[2]	[3]	[4]	[5]	TOP	OUTPUT
3							-1	3
+	+						0	3
1	+						0	31
*	+	*					1	31
2	+	*					1	312
+	+						0	312*+
(	+	(					1	312*+
4	+	(					1	312*+4
+	+	(	+				2	312*+4
5	+	(	+				2	312*+45
*	+	(	+	*			3	312*+45
6	+	(	+	*			3	312*+456
)	+						0	312*+456*+
/	+	/					1	312*+456*+
(	+	/	(				2	312*+456*+
9	+	/	(				2	312*+456*+9
-	+	/	(	-			3	312*+456*+9
7	+	/	(	-			3	312*+456*+97
)	+	/					1	312*+456*+97-
<eos>							-1	312*+456*+97-/+